Universidad Rafael Landívar

Facultad de Ingeniería

Licenciatura en Ingeniería en Informática y Sistemas

Lenguajes Formales y Autómatas Sección: 01

Docente: Ing. Cesar Rojas

PROYECTO NO.01

MOTOR DE PARSING, VALIDACIONES ESTRUCTURALES Y SIMULACIÓN DE AUTÓMATAS FINITOS DETERMINISTAS (AFD): ANÁLISIS Y DISEÑO

Estudiante: Gerber Mariano Péres Gómez,
Marco Fabrizzio Donadio Castellanos

Carné: 1165825, 1076925

Guatemala 30 de agosto de 2026

TABLA DE CONTENIDO

I. INTRODUCCIÓN 1
II. PLANTEAMIENTO DEL PROBLEMA, OBJETIVOS Y ALCANCE 1
2.1 Problema a resolver 1
2.2 Objetivo general 1
2.3 Objetivos específicos 2
2.4 Alcance 2
III. ARQUITECTURA GENERAL DEL SISTEMA 2
IV. ACCIONES DEL PROGRAMA 3
V. DATOS DE ENTRADA 4
5.1 Estructura obligatoria del archivo de definición 5
5.2 Expresiones regulares utilizadas 5
VI. VARIABLES DEL SISTEMA 5
6.1 Variables persistentes del AFD 6
6.2 Variables de control y auxiliares 6
VII. CONDICIONES, RESTRICCIONES Y CÁLCULOS 7
7.1 Condiciones formales de validez 7
7.2 Condiciones de simulación y aceptación 8
7.3 Cálculos de accesibilidad 8
7.4 Reglas de parsing y archivos 9
7.5 Recuperación mediante estado trampa 9
VIII. DISEÑO DEL ALGORITMO Y LÓGICA GENERAL 9
8.1 Algoritmo general del programa 9
8.2 Estrategia de validación antes de evaluar 10
8.3 Algoritmo de reparación 10
8.4 Persistencia y reemplazo 10
IX. EXPLICACIÓN TÉCNICA DE COMPONENTES 10
9.1 Clase AFD 10
9.2 Clase CargadorAFD 11
9.3 Clase ValidadorAFD 11
9.4 Clase SimuladorAFD 11
9.5 Funciones de coordinación 11
X. ANÁLISIS FORMAL DE COMPLEJIDAD 12
XI. BUENAS PRÁCTICAS Y DECISIONES DE INGENIERÍA 12
11.1 Precisión y límites vigentes 13
XII. DIAGRAMA DE FLUJO 14
XIII. CONCLUSIONES GENERALES 15

I. INTRODUCCIÓN

El proyecto Motor de Parsing, Validaciones Estructurales y Simulación de Autómatas Finitos Deterministas se desarrolló como una aplicación de consola orientada a representar y procesar formalmente un AFD a partir de su quíntupla $M = (Q, \Sigma, \delta, q0, F)$. La solución integra tres responsabilidades principales: la adquisición estructurada de la definición del autómata, la verificación de que dicha definición cumpla las propiedades teóricas de un AFD y la simulación de cadenas mediante el seguimiento explícito de la función de transición.

Desde la perspectiva del curso, el valor del programa no se limita a decidir si una cadena es aceptada o rechazada. El sistema también debe garantizar que el autómata pueda considerarse válido antes de procesarlo, señalar errores sintácticos sin colapsar, detectar inconsistencias como estados inexistentes o transiciones faltantes y producir una traza entendible del recorrido realizado. Por esta razón, el código se organizó en componentes separados para modelado, carga, validación, simulación e interacción mediante menú.

El presente documento expone el análisis y diseño del programa con énfasis en los elementos solicitados para la evaluación: acciones disponibles, datos de entrada, variables, condiciones y cálculos, y algoritmo completo. A su vez, se incorporan arquitectura, análisis de complejidad y decisiones de robustez con el propósito de que la solución pueda explicarse y defenderse técnicamente, siguiendo un enfoque consistente con una memoria de ingeniería.

II. PLANTEAMIENTO DEL PROBLEMA, OBJETIVOS Y ALCANCE

2.1 Problema a resolver

Un Autómata Finito Determinista puede expresarse de manera compacta mediante una quíntupla formal; sin embargo, esa representación solamente es procesable si existe coherencia entre sus componentes. Un estado inicial fuera de Q, un estado final inexistente, una transición que emplea un símbolo ajeno a Σ o dos destinos asociados al mismo par (estado, símbolo) invalidan la interpretación determinista del autómata. De igual forma, una definición textual mal formada no debe provocar que la aplicación termine abruptamente.

El problema computacional consiste, entonces, en construir una aplicación general capaz de recibir cualquier AFD que respete el formato establecido, convertir su definición textual en estructuras de datos, validar su integridad formal y, únicamente cuando corresponde, simular cadenas símbolo por símbolo. La solución debe funcionar sin modificar el código fuente para cada autómata distinto y debe mantener mensajes de error comprensibles durante toda la interacción.

2.2 Objetivo general

Diseñar e implementar un simulador de AFD en Python capaz de crear o cargar la quíntupla $M = (Q, \Sigma, \delta, q0, F)$, validar de manera estructural y sintáctica sus componentes y procesar cadenas individuales o por lote, generando la traza completa de ejecución y el veredicto de aceptación correspondiente.

2.3 Objetivos específicos

- Representar los estados y el alfabeto mediante conjuntos, y la función de transición mediante un diccionario indexado por la pareja (estado, símbolo).
- Permitir la construcción manual de un AFD completo mediante validaciones inmediatas sobre nombre, estados, alfabeto, estado inicial, estados finales y destinos de transición.
- Implementar un cargador de archivos .txt que utilice expresiones regulares para comprobar la sintaxis obligatoria de los encabezados y transiciones.
- Verificar $q0 \in Q$, $F \subseteq Q$, consistencia de δ, determinismo y completitud antes de simular cadenas.
- Identificar estados alcanzables, estados inaccesibles y estados finales alcanzables mediante una exploración DFS desde q0.
- Procesar cadenas símbolo por símbolo aplicando δ y registrar las evaluaciones válidas en un historial de sesión.
- Proporcionar un menú estable de diez opciones que permita repetir operaciones y reemplazar el autómata activo sin modificar el programa.

2.4 Alcance

El alcance comprende una aplicación en modo consola, creación manual, lectura de archivos de definición, evaluación individual y masiva de cadenas, visualización formal, tabla de transiciones, análisis estructural e historial de evaluaciones. No se utilizan bibliotecas que implementen autómatas finitos; la única importación externa a las capacidades básicas del lenguaje es el módulo estándar re, utilizado exclusivamente para validación mediante expresiones regulares. La persistencia se limita a los archivos de entrada: el historial existe únicamente durante la sesión de ejecución.

III. ARQUITECTURA GENERAL DEL SISTEMA

La solución se organizó mediante clases y funciones con responsabilidades diferenciadas. Esta separación evita concentrar la lógica de parsing, validación y simulación dentro del menú principal y permite que cada componente pueda explicarse de forma independiente. El objeto AFD funciona como modelo central en memoria; los demás componentes operan sobre él sin sustituir su responsabilidad.

Tabla 1. Componentes principales y responsabilidad dentro del programa

Componente	Responsabilidad principal
AFD	Representar la quíntupla formal y almacenar nombre, Q, Σ, q0, F, δ, transiciones repetidas e historial.
CargadorAFD	Construir el AFD manualmente o desde archivo; validar formato de conjuntos y sintaxis textual mediante expresiones regulares.

ValidadorAFD	Comprobar integridad estructural, determinismo, completitud y accesibilidad de estados.
SimuladorAFD	Procesar una cadena sobre δ, producir traza, determinar aceptación y registrar el resultado.
Funciones de interfaz	Coordinar validación, evaluación por lote, historial, reemplazo del autómata y presentación del menú.
main	Mantener afd_actual durante la sesión y dirigir el flujo de las diez opciones mediante un ciclo while.

La variable afd_actual actúa como punto de integración entre todas las capas. Al iniciar el programa su valor es None; únicamente después de una creación manual exitosa o una carga válida desde archivo pasa a referenciar un objeto AFD. Esta decisión permite que las opciones de visualización y evaluación detecten con claridad si existe información disponible antes de intentar utilizarla.

IV. ACCIONES DEL PROGRAMA

El menú principal ofrece diez acciones. Cada opción posee una responsabilidad concreta y, salvo la salida, retorna al menú principal para mantener una sesión interactiva continua. Las opciones que requieren un autómata verifican su existencia y, cuando se procesarán cadenas, también su validez estructural.

Tabla 2. Acciones disponibles en el menú principal

Opción	Acción	Precondición	Proceso principal	Resultado
1	Crear un AFD manualmente	Ninguna.	Solicita paso a paso nombre, Q, Σ, q0, F y una transición para cada pareja de Q × Σ.	Un nuevo AFD queda almacenado en afd_actual.
2	Cargar un AFD desde .txt	Ruta accesible y archivo con formato esperado.	Analiza encabezados y transiciones mediante regex; si la carga sintáctica finaliza, crea un AFD.	Reemplaza afd_actual solamente si cargar_archivo retorna un objeto.
3	Mostrar definición formal	Debe existir afd_actual.	Imprime nombre, Q, Σ, q0, F y cada transición δ(q,a)=q'.	Representación textual de la quíntupla.
4	Mostrar tabla de transiciones	Debe existir afd_actual; Q y Σ deben contener datos.	Ordena estados y símbolos y consulta δ para construir filas y columnas.	Tabla tabular; una transición ausente se muestra como ---.
5	Validar estructura	Debe existir afd_actual.	Ejecuta validación formal y, si es válida, DFS para accesibilidad.	Reporte VÁLIDO/INVÁLIDO, errores y análisis de estados.
6	Evaluar una cadena	Debe existir un AFD estructuralmente válido.	Valida símbolos, parte de q0, aplica δ por cada símbolo y comprueba el estado final.	Traza, Aceptada/Rechazada y registro en historial.
7	Evaluar archivo de cadenas	Debe existir un AFD estructuralmente válido y archivo accesible.	Lee cada línea y la envía al simulador con traza habilitada.	Resultados secuenciales para todas las líneas procesadas.
8	Consultar historial	Debe existir afd_actual.	Recorre los registros acumulados durante la sesión.	Cadena, estado final y resultado de cada evaluación registrada.

9	Cargar o crear otro autómata	Ninguna para abrir el submenú.	Permite ejecutar nuevamente creación manual o carga por archivo.	Si el nuevo AFD existe, reemplaza al anterior; si falla, conserva el actual.
10	Salir	Ninguna.	Muestra mensaje de despedida y ejecuta break.	Finaliza el ciclo principal y el programa.

V. DATOS DE ENTRADA

Las entradas del sistema se capturan inicialmente como texto porque input() retorna valores de tipo "str". Cuando la información representa conjuntos, el programa divide la cadena por comas, elimina espacios laterales y transforma los elementos válidos a "set". Esta diferencia entre formato de ingreso y representación interna es importante para comprender el diseño.

Tabla 3. Datos solicitados al usuario y tipo de dato asociado

Dato	Ejemplo de entrada	Tipo capturado	Representación interna	Validación principal
Opción	6	str	str	Debe coincidir con una cadena entre "1" y "10".
Nombr	AFD_AB	str	str	No puede quedar vacío después de strip().
Estados Q	q0,q1,q2	str	set[str]	No vacío; no se permiten elementos vacíos ni duplicados.
Alfabeto Σ	a,b	str	set[str]	No vacío; no se permiten elementos vacíos ni duplicados.
Estado inicial q0	q0	str	str	Debe pertenecer a Q.
Estados finales F	q2	str	set[str]	Puede ser vacío; si contiene elementos, todos deben pertenecer a Q.
Destino δ(q,a)	q1	str	str dentro del diccionario	Cada destino solicitado manualmente debe pertenecer a Q.
Ruta de definición	.\afd.txt	str	str	Debe poder abrirse en modo lectura UTF-8. Dentro de la raíz del proyecto.
Cadena individual	abba	str	str	Cada carácter procesado debe pertenecer a Σ.
Ruta de archivo de cadenas	.\cadenas.txt	str	str	Debe poder abrirse; cada línea se interpreta como una cadena. Dentro de la raíz del proyecto.
Opción del submenú de reemplazo	1	str	str	Solo se reconocen "1" y "2".

5.1 Estructura obligatoria del archivo de definición

La carga desde archivo utiliza un orden rígido para los seis primeros renglones. Las cinco primeras líneas deben contener los campos de la quíntupla y la sexta debe ser exactamente TRANSICIONES:. A partir de la séptima línea se procesa una transición por renglón.

```
NOMBRE=AFD_AB
ESTADOS=q0,q1,q2
ALFABETO=a,b
INICIAL=q0
FINALES=q2
TRANSICIONES:
q0,a,q1 q0,b,q0 q1,a,q1
q1,b,q2
```

Los encabezados se validan mediante patrones de expresiones regulares. NOMBRE, ESTADOS, ALFABETO e INICIAL requieren contenido después del signo igual; FINALES permite un valor vacío. Cada transición debe cumplir el patrón origen,símbolo,destino sin espacios internos, comas adicionales ni campos vacíos.

5.2 Expresiones regulares utilizadas

Los encabezados se validan en orden. NOMBRE, ESTADOS, ALFABETO e INICIAL requieren contenido; FINALES admite el conjunto vacío. La transición exige exactamente tres campos sin espacios internos.

```
^NOMBRE=(?P<valor>.+)$
^ESTADOS=(?P<valor>.+)$
^ALFABETO=(?P<valor>.+)$
^INICIAL=(?P<valor>.+)$
^FINALES=(?P<valor>.*)$
^(?P<origen>[^,\s]+),(?P<simbolo>[^,\s]+),(?P<destino>[^,\s]+)$
```

VI. VARIABLES DEL SISTEMA

Para evitar un listado mecánico de todas las variables temporales, se distinguen las variables persistentes del modelo y las variables auxiliares que determinan el flujo de los algoritmos. Las primeras describen el estado del AFD durante la sesión; las segundas existen durante operaciones concretas de carga, validación o simulación.

6.1 Variables persistentes del AFD

Tabla 4. Variables principales almacenadas por el objeto AFD

Variable	Tipo conceptual	Propósito
nombre	str	Identificador textual del autómata.
estados	set[str]	Conjunto Q de todos los estados declarados.
alfabeto	set[str]	Conjunto Σ de símbolos válidos.
estado_inicial	str	Estado q0 desde el cual comienza toda simulación.
estados_finales	set[str]	Conjunto F de estados de aceptación.
transiciones	diccionario[tuple[str,str], str]	Función δ; la clave es (estado_origen, símbolo) y el valor es el estado_destino.
transiciones_repetidas	list[tuple[str,str,str]]	Registro de intentos de definir más de una transición para el mismo par (estado, símbolo).
historial	list[diccionario]	Registros de cadena, estado final y resultado de las evaluaciones terminadas.
es_valido	bool	Bandera calculada por ValidadorAFD.validar() que refleja si no se encontraron errores estructurales.

6.2 Variables de control y auxiliares

Tabla 5. Variables relevantes durante la ejecución

Variable	Contexto	Tipo	Función
afd_actual	main	AFD | None	Referencia al autómata activo de la sesión.
opcion	menús	str	Determina la rama del flujo que debe ejecutarse.

errores	carga / validación	list[str]	Acumula problemas para reportarlos conjuntamente sin colapsar.
clave	transiciones	tuple[str,str]	Representa la pareja (estado_origen, símbolo).
estado_destino / destino	carga / δ	str	Estado alcanzado por una transición.
estados_alcanzables	análisis estructural	set[str]	Conjunto R de estados visitados desde q0.
pendientes	DFS	list[str]	Pila LIFO de estados todavía esperando por ser visitados.
inaccesibles	análisis estructural	set[str]	Diferencia Q − R.
finales_alcanzables	análisis estructural	set[str]	Intersección F ∩ R.
cadena	simulación	str	Secuencia que se procesa símbolo por símbolo.
estado_actual	simulación	str	Estado vigente durante el recorrido.
siguiente_estado	simulación	str	Resultado de δ(estado_actual, símbolo).
aceptada	simulación	bool	Indica si el estado final pertenece a F.
resultado	simulación	str	Cadena descriptiva: Aceptada, Rechazada o mensaje de error.
lineas / limpias	carga de archivos	list[str]	Contenido leído y versión normalizada con strip().
valores_encabezado	parser	diccionario[str,str]	Almacena el valor extraído por regex para cada encabezado.
coincidencia	parser	re.Match | None	Indica si una línea cumple el patrón requerido.
nuevo_afd	opciones 2 y 9	AFD | None	Permite reemplazar el autómata actual únicamente cuando la operación fue exitosa.

VII. CONDICIONES, RESTRICCIONES Y CÁLCULOS

El proyecto no realiza cálculos aritméticos complejos; su núcleo matemático está compuesto por relaciones de pertenencia, operaciones de conjuntos, evaluación de una función de transición y comprobaciones de cardinalidad implícita. Estas condiciones son las que convierten una definición textual en un AFD estructuralmente válido.

7.1 Condiciones formales de validez

Tabla 6. Reglas estructurales aplicadas al AFD

Regla	Expresión / criterio	Aplicación en el programa
Existencia de componentes	nombre ≠ "", Q ≠ ∅, Σ ≠ ∅	El validador reporta error cuando falta el identificador o alguno de los conjuntos básicos está vacío.
Estado inicial válido	$q_0 \in Q$	El estado inicial debe ser uno de los estados declarados.

Estados finales válidos	F⊑ Q	Cada estado de aceptación debe pertenecer a Q; F puede ser∅.
Consistencia de transición	q ⊢ ⊢Q, a Σ, q' ⊢Q	Para toda transición δ(q,a)=q', origen, símbolo y destino deben existir en sus conjuntos correspondientes.
Determinismo	Para cada (q,a) existe como máximo un destino	Una segunda definición de la misma clave se registra en transiciones_repetidas y produce error de validación.
Completitud	Para todo q⊢ Q y a⊢ Σ debe existir δ(q,a)	El validador recorre Q × Σ y reporta cada transición faltante.
Cardinalidad esperada de δ	|δ| = |Q| · |Σ| para un AFD completo y determinista	Consecuencia de disponer de exactamente una transición por cada combinación de estado y símbolo.

7.2 Condiciones de simulación y aceptación

Sea una cadena $w = a_1 a_2 \ldots a_n$. La simulación se inicia con estado_actual = q0. Para cada símbolo a_i se actualiza el estado según la relación estado_actual ← $\delta(estado_actual\ ,\ a_i)$. Debido a que δ se almacena en un diccionario, el programa localiza el siguiente estado mediante la clave (estado_actual, símbolo).

Antes de ejecutar las transiciones se verifica que cada símbolo procesado pertenezca a Σ. Si aparece un símbolo inválido, la función retorna un mensaje de error y esa evaluación no se incorpora al historial. Si la cadena se procesa completamente, el veredicto se obtiene con la condición $q_final \in$ F: si es verdadera, la cadena se considera aceptada; de lo contrario, rechazada.

La cadena vacía también puede representarse mediante un string de longitud cero. En ese caso no se ejecuta ninguna transición y el estado final coincide con q0; por consiguiente, se acepta únicamente cuando q0 ∈ F. Este comportamiento se deriva directamente de la lógica implementada, sin requerir un caso especial.

7.3 Cálculos de accesibilidad

Después de que el AFD supera la validación estructural, el sistema analiza qué estados son realmente alcanzables. Se ejecuta una búsqueda en profundidad utilizando una lista como pila LIFO. A partir del estado inicial se exploran los destinos disponibles para todos los símbolos del alfabeto y se evita procesar más de una vez un estado ya visitado.

Tabla 7. Operaciones de conjuntos utilizadas en el análisis estructural

Resultado	Expresión	Interpretación
Estados alcanzables	R = DFS(q0)	Estados que pueden visitarse siguiendo cero o más transiciones desde q0.
Estados inaccesibles	Q − R	Estados declarados pero nunca alcanzados desde el inicio.
Finales alcanzables	F ∩ R	Estados de aceptación que pueden aparecer al finalizar alguna ruta alcanzable.
Posible lenguaje vacío	F ∩ R = ∅	El programa advierte que no existe un estado final alcanzable desde q0.

7.4 Reglas de parsing y archivos

- El archivo de definición debe contener al menos seis líneas.
- Los encabezados deben aparecer en el orden NOMBRE, ESTADOS, ALFABETO, INICIAL y FINALES.
- La sexta línea debe ser exactamente TRANSICIONES:.
- No se permiten líneas vacías dentro de la sección de transiciones.
- Cada transición debe tener exactamente tres campos sin espacios: origen,símbolo,destino.
- ESTADOS y ALFABETO no pueden quedar vacíos; FINALES sí puede quedar vacío.
- Los conjuntos declarados no pueden contener elementos duplicados.
- Una carga sintácticamente correcta puede todavía producir un AFD estructuralmente inválido; esa segunda capa de comprobación corresponde a ValidadorAFD.

7.5 Recuperación mediante estado trampa

La reparación se ofrece únicamente cuando el número de errores coincide con el número de transiciones faltantes; por tanto, no se ejecuta si existen ε, estados inválidos, símbolos externos o no determinismo. Se calcula la lista de parejas ausentes antes de incorporar el nuevo estado, se elige un nombre disponible (q_trampa, q_trampa_1, etc.), se dirigen las transiciones faltantes hacia él y se agregan ciclos $\delta(q_trampa,a)=q_trampa$ para todo $a \in \Sigma$. Finalmente, la bandera es_valido se establece en False y el AFD se valida nuevamente.

VIII. DISEÑO DEL ALGORITMO Y LÓGICA GENERAL

La función main mantiene un ciclo while True que representa la vida completa de la sesión. En cada iteración se imprime el menú, se captura la opción como texto y se ejecuta una rama if/elif. Todas las ramas operativas regresan posteriormente al inicio del ciclo; la única excepción es la opción 10, que ejecuta break y finaliza el programa. El estado compartido entre iteraciones es afd_actual.

8.1 Algoritmo general del programa

1. Inicializar afd_actual = None.
2. Mostrar el mensaje de bienvenida.
3. Entrar en un ciclo while True.
4. Mostrar el menú y solicitar una opción entre 1 y 10.
5. Ejecutar la acción correspondiente. Si la operación necesita un AFD, comprobar primero que exista; si se procesarán cadenas, ejecutar además la validación estructural.
6. Cuando se crea o carga un nuevo AFD con éxito, asignarlo a afd_actual. Si la carga falla, no sustituir una referencia previa válida.
7. Después de terminar la operación, regresar al menú principal.

8. Si la opción es 10, mostrar el mensaje de salida, romper el ciclo y finalizar.

8.2 Estrategia de validación antes de evaluar

Una decisión importante del flujo es que el usuario no está obligado a ejecutar manualmente la opción 5 antes de utilizar las opciones 6 o 7. La función asegurar_afd_valido vuelve a ejecutar ValidadorAFD.validar() y detiene la operación cuando encuentra errores. De esta manera, la validez estructural funciona como precondición del procesamiento y no únicamente como una acción informativa del menú.

8.3 Algoritmo de reparación

1. Obtener todas las parejas (q,a) ausentes mediante obtener_transiciones_faltantes.
2. Comprobar que no existan errores distintos de la incompletitud.
3. Solicitar autorización explícita al usuario.
4. Generar un nombre de estado trampa que no colisione con Q.
5. Agregar el estado y dirigir hacia él cada transición faltante.
6. Agregar un ciclo en el estado trampa para cada símbolo de Σ.
7. Invalidar la bandera anterior, validar nuevamente y mostrar el reporte actualizado.

8.4 Persistencia y reemplazo

Las opciones 2 y 9 utilizan nuevo_afd antes de sustituir afd_actual, de modo que una carga fallida no elimina un autómata previamente válido. El historial pertenece a cada objeto AFD y permanece solo durante la sesión.

IX. EXPLICACIÓN TÉCNICA DE COMPONENTES

9.1 Clase AFD

La clase AFD representa el dominio formal del problema. Los conjuntos Q, Σ y F se almacenan mediante set, lo cual facilita pruebas de pertenencia y operaciones como diferencia e intersección. La función δ se almacena en un diccionario donde cada clave es una tupla (estado, símbolo). Esta decisión evita construir una cadena extensa de condicionales para un autómata específico y hace que la solución sea reutilizable con cualquier definición válida.

El método ***agregar_transicion*** centraliza la inserción en δ. Antes de escribir la nueva pareja, consulta si la clave ya existe. Si existe, no sobrescribe silenciosamente el destino anterior; registra la anomalía en ***transiciones_repetidas*** y retorna False. Esta estrategia conserva evidencia del no-determinismo para que la capa de validación pueda reportarlo posteriormente.

Los métodos ***mostrar_definicion_formal*** y ***mostrar_tabla_transicion*** corresponden a la capa de visualización. El primero recorre el diccionario y muestra cada transición con notación δ(q,a)=q'. El segundo ordena estados y símbolos para producir una salida tabular reproducible y utiliza el marcador "--" cuando no existe una transición, lo cual permite visualizar un AFD parcial antes de validar.

9.2 Clase CargadorAFD

CargadorAFD implementa dos rutas de adquisición. La creación manual controla el ingreso inmediatamente: solicita conjuntos sin duplicados, exige $q0 \in Q$, exige $F \subseteq Q$ y pregunta explícitamente un destino para cada combinación $Q \times \Sigma$. Por construcción, esta ruta produce una función de transición completa y evita definir dos veces la misma pareja.

La carga desde archivo separa parsing sintáctico y validación formal. Primero se intenta abrir el archivo y se normalizan sus líneas. Luego se validan los encabezados mediante expresiones regulares, se convierten las listas separadas por comas y finalmente se analizan las transiciones. Los errores de sintaxis se acumulan y se muestran con número de línea cuando corresponde; si existen, el método retorna None en lugar de provocar una excepción no controlada.

9.3 Clase ValidadorAFD

ValidadorAFD verifica que la representación almacenada pueda considerarse un AFD válido. La rutina validar no se limita a comprobar campos existentes: recorre F, revisa cada transición registrada, incorpora las transiciones repetidas al reporte y ejecuta un doble recorrido $Q \times \Sigma$ para detectar cualquier transición faltante. El resultado se devuelve como una tupla compuesta por una bandera booleana y la lista completa de errores.

El método ***analizar_estructura*** se ejecuta después de una validación exitosa. Utiliza DFS con una pila explícita, marca estados visitados y explora sus vecinos consultando δ para cada símbolo de Σ. Con el conjunto resultante calcula estados inaccesibles y estados finales alcanzables. Esta etapa no modifica el AFD; produce un diagnóstico adicional sobre su estructura efectiva.

9.4 Clase SimuladorAFD

SimuladorAFD concentra la semántica de reconocimiento de cadenas. Primero requiere que el atributo ***es_valido*** indique una validación positiva y luego verifica que la cadena no contenga símbolos externos al alfabeto. La simulación comienza en q0 y avanza consultando directamente el diccionario de transiciones. Cuando ***mostrar_traza*** es True, cada cambio de estado se imprime antes de actualizar la variable ***estado_actual***.

Una evaluación terminada genera un registro en historial con tres campos: cadena, estado_final y resultado. Los intentos que fallan por símbolo inválido retornan antes de llegar a esta sección, por lo que el historial representa evaluaciones completadas y no simplemente entradas intentadas.

9.5 Funciones de coordinación

mostrar_validacion presenta el resultado del validador y agrega el análisis de accesibilidad cuando el AFD es válido. ***asegurar_afd_valido*** se utiliza como guardia común para las opciones de evaluación. ***evaluar_archivo_cadenas*** procesa un archivo completo línea por línea; ***mostrar_historial*** recorre los registros acumulados; ***crear_o_cargar_otro*** encapsula el submenú de reemplazo; ***mostrar_menu*** centraliza la interfaz; y ***main*** mantiene el flujo general.

X. ANÁLISIS FORMAL DE COMPLEJIDAD

Aunque el enunciado no exige un análisis asintótico, incluirlo permite justificar el costo de las operaciones principales y relacionar directamente las estructuras de datos elegidas con el comportamiento del programa. Se considera acceso promedio O(1) para set y dict (diccionario) en Python.

Tabla 9. Complejidad temporal de operaciones principales

Operación	Complejidad	Justificación
Agregar transición	O(1) promedio	La comprobación de existencia y la inserción se realizan sobre un diccionario por clave (q,a).
Creación manual de δ	O(|Q|·|Σ|)	Se solicita exactamente una transición para cada combinación de estado y símbolo.
Mostrar tabla de transiciones	O(|Q|·|Σ|)	Se recorre cada celda conceptual Q × Σ y se consulta el diccionario.
Validación estructural	O(|δ| + |F| + |Q|·|Σ|)	Se revisan transiciones existentes, estados finales y completitud sobre todo Q × Σ.
Análisis de accesibilidad	O(|R|·|Σ|)	Cada estado alcanzable se procesa una vez y para él se consultan todos los símbolos del alfabeto.
Evaluar una cadena w	O(|w|)	Se valida y procesa la cadena linealmente; cada transición se consulta en el diccionario.
Evaluar archivo de cadenas	O(Σ |wi|)	El costo dominante es la suma de las longitudes de todas las cadenas leídas.
Mostrar historial	O(h)	Se imprime una vez cada uno de los h registros de evaluación.
Parsing de archivo	O(L)	Las líneas se recorren en orden y cada una se valida una cantidad constante de veces, ignorando la longitud particular de cada renglón.

En términos de espacio, el AFD requiere O(|Q| + |Σ| + |δ| + |F| + h) para almacenar sus componentes y el historial. El análisis DFS utiliza además un conjunto de visitados y una pila cuyo tamaño está acotado por O(|Q|).

XI. BUENAS PRÁCTICAS Y DECISIONES DE INGENIERÍA

La robustez del programa depende tanto de la teoría del AFD como de la forma en que se gestionan entradas y errores. La implementación incorpora varias decisiones que reducen estados inconsistentes y hacen más defendible el comportamiento del sistema.

Tabla 10. Decisiones de robustez observables en la implementación

Decisión	Manifestación concreta	Beneficio

Separación de responsabilidades	AFD, CargadorAFD, ValidadorAFD y SimuladorAFD tienen funciones distintas.	Evita mezclar parsing, modelo formal, validación y ejecución.
Estructuras apropiadas	set para Q/Σ/F y dict para δ.	Permite pertenencia eficiente y modela naturalmente la función de transición.
Validación por capas	Primero sintaxis del archivo; después integridad formal del AFD.	Distingue un archivo mal formado de un autómata conceptualmente inválido.
Acumulación de errores	Las listas errores reúnen múltiples fallos antes de reportar.	El usuario recibe un diagnóstico más útil que el primer error aislado.
Protección del estado actual	nuevo_afd se asigna antes de reemplazar afd_actual.	Una carga fallida no destruye un autómata previo disponible.
Prevención de no-determinismo	agregar_transicion detecta claves repetidas.	Evita sobrescrituras silenciosas y conserva evidencia para el validador.
Precondición automática de evaluación	asegurar_afd_valido se ejecuta antes de opciones 6 y 7.	Impide simular un AFD incompleto o inconsistente.
Trazabilidad	SimuladorAFD imprime estado actual, símbolo y siguiente estado.	Facilita comprobar manualmente por qué una cadena fue aceptada o rechazada.
Historial de sesión	Cada evaluación completada se registra.	Permite revisar resultados sin repetir inmediatamente las cadenas.
Manejo de archivos	Los accesos se rodean con try/except y retornos controlados.	Evita que rutas inexistentes o problemas de lectura finalicen el programa abruptamente.

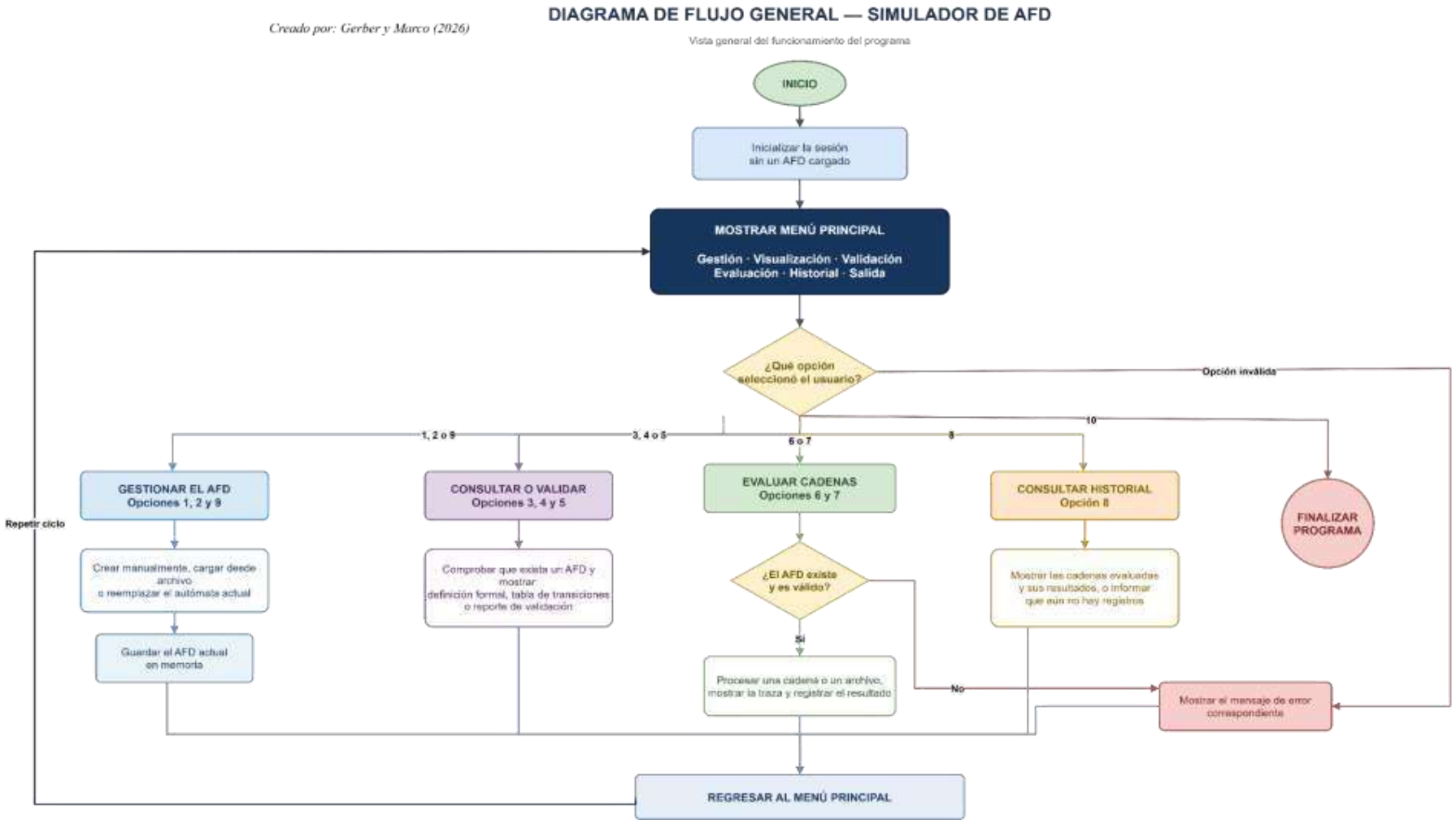
11.1 Precisión y límites vigentes

La implementación procesa alfabetos cuyos símbolos se representan mediante exactamente un carácter; esta condición ya no constituye una limitación implícita, sino una regla validada tanto en la creación manual como en el archivo. El programa distingue la cadena vacía —entrada de longitud cero— de una transición ε, que se rechaza. La persistencia continúa limitada a los archivos de entrada y el historial se conserva únicamente durante la sesión.

XII. DIAGRAMA DE FLUJO

Para cumplir el apartado de algoritmo se construyó un diagrama de flujo maestro que representa el ciclo completo del menú. Esta vista resume la inicialización, lectura de opción, selección de la acción y retorno al menú después de cada operación. La opción 10 constituye la única salida del ciclo.

Figura 1. Diagrama de flujo general del programa para la fase 1



XIII. CONCLUSIONES GENERALES

Se concluye que la solución modela de manera directa la quíntupla formal de un AFD y separa adecuadamente las responsabilidades necesarias para convertir una definición textual en una simulación confiable. El uso de conjuntos para Q, Σ y F y de un diccionario para δ mantiene una correspondencia clara entre la teoría y la estructura computacional utilizada.

La validación constituye el punto central de robustez del proyecto. Antes de procesar una cadena se comprueba pertenencia del estado inicial, consistencia de estados finales, validez de cada transición, determinismo y completitud. Además, la exploración DFS permite diferenciar estados declarados de estados efectivamente alcanzables y aporta una lectura estructural que trasciende el simple veredicto de aceptación.

La carga desde archivo fortalece el proyecto al separar errores sintácticos de errores formales. Las expresiones regulares verifican el patrón de entrada, mientras que ValidadorAFD analiza posteriormente la semántica del autómata. Esta separación permite controlar fallos sin colapsar y producir mensajes útiles para el usuario.

Finalmente, el sistema de menú integra creación, visualización, validación, simulación e historial dentro de una sesión continua. Los diagramas de flujo muestran que la lógica global puede resumirse en un ciclo principal, pero también evidencian que las opciones de creación, parsing, validación y evaluación contienen suficientes decisiones internas para justificar representaciones específicas cuando se busca máxima claridad durante la defensa técnica.

Universidad Rafael Landívar

Facultad de Ingeniería

Licenciatura en Ingeniería en Informática y Sistemas

Lenguajes Formales y Autómatas Sección: 01

Docente: Ing. Cesar Rojas

PROYECTO NO.02

MOTOR DE PARSING, VALIDACIONES ESTRUCTURALES Y SIMULACIÓN DE AUTÓMATAS FINITOS NO DETERMINISTAS (AFND): ANÁLISIS Y DISEÑO

Estudiante: Gerber Mariano Péres Gómez,
Marco Fabrizzio Donadio Castellanos

Carné: 1165825, 1076925

Guatemala 25 de septiembre de 2026

TABLA DE CONTENIDO

I. INTRODUCCIÓN

La Fase 2 amplía el motor de autómatas para representar Autómatas Finitos No Deterministas sin transiciones epsilon, validar su estructura y convertirlos a un AFD equivalente mediante construcción de subconjuntos. La aplicación conserva las funciones de la fase anterior y agrega un flujo específico para trabajar con múltiples destinos por transición.

El cambio central consiste en que la función de transición del AFND asocia cada pareja de estado y símbolo con un conjunto de destinos. Después de crear o cargar un AFND válido, el programa muestra su tabla y genera automáticamente el AFD equivalente. La opción de conversión permanece disponible en el menú para confirmar el proceso o reconstruir el resultado si fuera necesario.

Las evaluaciones de cadenas se resuelven con el AFD operativo. Cuando el usuario cargó un AFND, el programa selecciona su AFD equivalente e informa esta decisión antes de simular. Debido a la equivalencia de lenguajes establecida por el algoritmo, el veredicto corresponde tanto al AFND original como al AFD generado.

II. ALCANCE Y CAMBIOS DE LA FASE 2

La implementación incorpora las clases AFND, CargadorAFND, ValidadorAFND y ConvertidorAFND. También actualiza el menú principal para conservar simultáneamente el autómata original, el AFD generado y la relación entre macroestados. No se implementan transiciones epsilon; cada símbolo del alfabeto debe ser un único carácter visible.

- Crear un AFND manualmente o cargarlo desde un archivo de texto.
- Aceptar cero, uno o varios destinos por pareja estado-símbolo en las transiciones.
- Representar varios destinos con una barra vertical y la ausencia de destino con ∅ o {}.
- Validar referencias, tipos, alfabeto, estado inicial, estados finales y destinos.
- Construir automáticamente AFD equivalente y conservar la tabla de equivalencias.
- Consultar por separado el AFND original y el AFD generado.
- Evaluar cadenas, archivos de cadenas, historial y estructura mediante el AFD operativo.

III. ARQUITECTURA ACTUALIZADA

La arquitectura mantiene responsabilidades separadas. El objeto original permanece disponible para consulta, mientras que la conversión crea un segundo objeto AFD sin modificar el AFND. Esta separación permite explicar el proceso y evita que la simulación mezcle estructuras no deterministas con la lógica determinista ya validada en la fase anterior.

Tabla 1: Componentes incorporados o actualizados en la Fase 2

Componente	Responsabilidad
AFND	Representa Q, Σ, q0, F y δ, donde cada valor de δ es un conjunto de estados destino.
CargadorAFND	Crea el AFND por consola o procesa el formato de archivo con TIPO=AFND y destinos separados por |.
ValidadorAFND	Comprueba tipos, nombres, pertenencia, símbolos, ausencia de epsilon y validez de todos los destinos.
ConvertidorAFND	Aplica construcción de subconjuntos, crea macroestados, transiciones, finales y equivalencias.
AFD y SimuladorAFD	Reutilizan la validación, traza, aceptación, historial, análisis y estado trampa de la Fase 1.
main	Mantiene automata_actual, afd_generado, equivalencias y definicion_convertida durante la sesión.

3.1 Estados de la sesión

automata_actual conserva el AFD o AFND cargado por el usuario**.** ***afd_generado*** solo contiene un valor cuando el original es un AFND validado y convertido. ***equivalencias*** relaciona cada identificador del AFD con su subconjunto de estados del AFND. ***definicion_convertida*** guarda una copia de los componentes y permite descartar una conversión obsoleta si el AFND cambia directamente.

Una carga fallida o una conversión con errores no sustituye la sesión anterior. El reemplazo se confirma únicamente después de preparar correctamente el nuevo autómata y, en el caso del AFND, después de generar su AFD equivalente.

IV. ACCIONES DEL MENÚ

El menú creció de diez a quince opciones. Las acciones de AFD se mantienen y se integran con las nuevas operaciones de AFND, conversión y consulta de macroestados.

Tabla 2: Acciones disponibles en el menú de la Fase 2

Opción	Acción	Cambio	Resultado
1	Crear AFD manualmente	Conservada	Crea un AFD y limpia los datos de la sesión anterior.
2	Cargar AFD desde archivo	Conservada	Lee y valida sintaxis de un AFD almacenado en texto.
3	Crear AFND manualmente	Nueva	Solicita conjuntos de destinos para cada pareja de transición.
4	Cargar AFND desde archivo	Nueva	Procesa TIPO=AFND y destinos separados por barra vertical.
5	Mostrar autómata original	Actualizada	Muestra definición y tabla del AFD o AFND cargado.
6	Validar original y generado	Nueva	Valida el original y, si existe, también el AFD equivalente.
7	Convertir AFND a AFD	Nueva	Confirma o reconstruye la conversión; reutiliza una versión válida.
8	Mostrar equivalencias	Nueva	Relaciona A, B, C y demás macroestados con subconjuntos del AFND.
9	Mostrar AFD generado	Nueva	Presenta la definición formal y tabla del AFD equivalente.
10	Evaluar una cadena	Actualizada	Usa el AFD directo o el AFD equivalente del AFND.
11	Evaluar archivo de cadenas	Actualizada	Procesa cada línea y conserva epsilon como línea vacía.
12	Consultar historial	Actualizada	Muestra el historial del AFD operativo de la sesión.
13	Análisis estructural	Actualizada	Calcula alcanzables, inaccesibles, finales útiles y lenguaje vacío.
14	Cargar o crear otro autómata	Actualizada	Abre un submenú con las cuatro formas de ingreso.

15	Salir	Renumerada	Finaliza el ciclo principal de manera controlada.

V. DATOS DE ENTRADA DEL AFND

El AFND se define mediante la misma quíntupla M = (Q, Σ, δ, q0, F), pero δ(q,a) devuelve un subconjunto de Q. Los encabezados pueden aparecer antes de la sección TRANSICIONES y cada clave debe existir una sola vez. El archivo se lee con UTF-8 con soporte para marca BOM.

Tabla 3 Entradas específicas del AFND

Dato	Ejemplo	Representación	Regla Principal
Tipo	TIPO=AFND	str	El valor debe ser exactamente AFND sin importar mayúsculas.
Estados	q0,q1,q2	set[str]	No vacíos, sin duplicados ni nombres reservados.
Alfabeto	a,b	set[str]	Símbolos individuales, visibles y distintos de epsilon.
Destinos múltiples	q0|q1	set[str]	La barra | separa destinos de una misma pareja
Sin destino	∅ o {}	set()	No puede combinarse con otros destinos
Finales	q2 o vacío	set[str]	Todos deben pertenecer a Q; el conjunto puede ser vacío.

5.1 Formato del Archivo

```
NOMBRE=AFND_AB
TIPO=AFND
ESTADOS=q0,q1,q2
ALFABETO=a,b
INICIAL=q0
FINALES=q2
TRANSICIONES:
q0,a,q0|q1
q0,b,q0
```

```
q1,a,∅
q1,b,q2
```

En q0,a,q0|q1 la barra no crea dos transiciones separadas: expresa un único valor δ(q0,a) = {q0,q1}. En q1,a,∅ la transición no tiene destinos. Si este vacío se alcanza durante la conversión, se crea un macroestado que representa ∅ y funciona como estado trampa del AFD equivalente.

5.2 Reglas de entrada

- La coma separa origen, símbolo y conjunto de destinos; la barra separa destinos dentro del tercer campo.
- No se aceptan campos vacíos entre barras, destinos duplicados ni combinaciones como q0|∅.
- Los nombres de estado no pueden contener espacios, caracteres de control, comas ni barras.
- Los nombres ∅ y {} están reservados para representar el conjunto vacío.
- FINALES= y una sección TRANSICIONES sin renglones posteriores son válidos para un AFND.

VI. VARIABLES Y ESTRUCTURAS

Tabla 4: Variables principales de la Fase 2

Variable	Tipo	Uso
transiciones	dict[(str,str), set[str]]	Asocia cada pareja del AFND con cero o más destinos.
automata_actual	AFD | AFND| None	Conserva la definición original activa.
afd_generado	AFD| None	Almacena el resultado de la construcción de subconjuntos.
equivalencias	dict[(str, set[str]]	Relaciona cada macroestado con los estados del AFND que representa.
definición_convertida	tuple | None	Detecta cambios posteriores en el AFND original.
subconjunto_a_nombre	dict[frozenset,str]	Evita crear dos nombres para el mismo subconjunto.
pendientes	list[frozenset]	Cola de macroestados todavía no procesados.
union_destinos	set[str]	Une los destinos alcanzables desde un macroestado con un símbolo.

frozenset se utiliza como clave porque un set normal es mutable y no puede indexar un diccionario. Las ***equivalencias*** conservan sets para mostrarlos con notación de conjuntos. El AFD generado vuelve a usar destinos simples de tipo ***str***, por lo que puede ser validado y simulado con las clases existentes.

VII. VALIDACIÓN DEL AFND

ValidadorAFND revisa primero los tipos de las estructuras para evitar recorridos inseguros. Después aplica las condiciones formales del AFND. La función de transición puede ser parcial y puede producir el conjunto vacío; por esta razón no se exige una transición no vacía para cada pareja de $Q \times \Sigma$.

Tabla 5: Condiciones de validez del AFND

Condición	Criterio	Comportamiento
Componentes	Q, Σ y F son conjuntos; δ es diccionario	Los tipos inválidos detienen la validación estructurada.
Nombre	nombre no vacío	Identifica el AFND durante consultas y conversión.
Estado inicial	$q0 \in Q$	El punto de partida debe estar declarado.
Estados finales	$F \subseteq Q$	F puede ser vacío.
Alfabeto	$a \in \Sigma$ tiene longitud 1	Se rechazan espacios, controles y epsilon.
Transición	$q \in Q$, $a \in \Sigma$, $\delta(q,a) \subseteq Q$	Cada destino se valida por separado.
Parcialidad	$\delta(q,a)$ puede ser $\emptyset$	La ausencia de camino es válida en el AFND.

La creación y la carga ejecutan comprobaciones inmediatas, pero ValidadorAFND constituye la verificación formal definitiva. Si encuentra errores, la conversión automática no se ejecuta y el autómata anterior permanece activo.

VIII. CONVERSIÓN AUTOMÁTICA A AFD

preparar_nuevo_automata se ejecuta inmediatamente después de crear o cargar un AFND. Si la validación es positiva, muestra la tabla del AFND y llama a ***ConvertidorAFND.convertir***. Por ello el AFD equivalente queda disponible desde el inicio de la nueva sesión, aunque la opción 7 continúa visible para confirmar o repetir el procedimiento.

8.1 Construcción de subconjuntos

1. Crear el macroestado inicial $A = \{q0\}$ y colocarlo en la cola pendientes.
2. Tomar un macroestado pendiente y recorrer cada símbolo del alfabeto.
3. Unir los destinos de todos los estados que integran el macroestado.
4. Convertir la unión en frozenset y asignarle A, B, C, hasta AA y los identificadores siguientes si aún no existe.
5. Agregar al AFD la transición hacia el identificador del subconjunto calculado.
6. Marcar como final cada macroestado cuya intersección con F no sea vacía.
7. Validar el AFD generado para confirmar determinismo, completitud y referencias correctas.

8.2 Macroestado vacío

Cuando la unión de destinos es vacía se registra frozenset() como un macroestado normal. Para cualquier símbolo, la unión de destinos desde ese conjunto vuelve a ser vacía; por tanto, el estado posee ciclos hacia sí mismo y actúa como estado trampa. No debe confundirse con los estados separados por |, que son destinos válidos del AFND.

8.3 Preservación de datos

La opción 7 reutiliza un AFD generado que todavía sea válido. Esta decisión conserva el historial y la tabla de equivalencias. Si el AFND cambia, la copia almacenada en definición_convertida deja de coincidir, el programa descarta la conversión anterior y exige generar una nueva.

IX. EVALUACIÓN Y ANÁLISIS

obtener_afd_operativo selecciona el AFD cargado directamente o el AFD equivalente. Antes de cada simulación, ***asegurar_afd_valido*** ejecuta de nuevo ***ValidadorAFD.validar*** para impedir evaluaciones sobre una estructura incompleta o modificada.

SimuladorAFD inicia en q0, consume la cadena símbolo por símbolo y consulta δ(estado_actual, símbolo). La cadena se acepta cuando el estado final pertenece a F. Cada evaluación terminada se agrega al historial con la cadena, el estado final y el resultado. Un símbolo ajeno al alfabeto produce un error y no agrega un registro.

Tabla 6: Relación entre AFND y AFD durante las pruebas

Acción	Estrucutra utilizada	Justificación
Mostrar original	AFND	Permite revisar los destinos múltiples y los conjuntos vacíos.
Mostrar equivalentes	AFND Y AFD	Explica qué subconjunto representa cada macroestado.
Evaluar cadena	AFD equivalente	La simulación determinista recorre una transición por símbolo.
Evaluar lote	AFD equivalente	Cada línea sigue el mismo proceso y comparte historial.
Análisis estructural	a ∈ Σ tiene longitud 1	Calcula alcanzables e inaccesibles sobre el autómata operativo.

Aunque la ejecución se realice sobre el AFD, la prueba representa también al AFND porque la construcción de subconjuntos preserva el lenguaje. Las opciones 6, 8 y 9 permiten comprobar de forma explícita la validez y la correspondencia antes de evaluar.

X. EXPLICACIÓN TÉCNICA DE COMPONENTES

Aunque la ejecución se realice sobre el AFD, la prueba representa también al AFND porque la construcción de subconjuntos preserva el lenguaje. Las opciones 6, 8 y 9 permiten comprobar de forma explícita la validez y la correspondencia antes de evaluar.

10.1 Clase AFND

AFND almacena los destinos como set. agregar_transicion une conjuntos cuando recibe nuevamente una pareja, invalida una validación previa y evita exponer referencias mutables. obtener_destinos retorna una copia y entrega set() cuando la pareja no fue definida. Las funciones de presentación recorren Q × Σ para mostrar explícitamente cada conjunto, incluido ∅.

10.2 Clase CargadorAFND

La creación manual solicita nombre, Q, Σ, q0 y F, y después pregunta los destinos para cada combinación. convertir_destinos interpreta la barra vertical, detecta campos vacíos y duplicados, y transforma ∅ o {} en set(). La carga por archivo acumula errores con número de línea y solo construye el objeto cuando la sintaxis y las referencias son coherentes.

10.3 Clase ValidadorAFND

El validador reutiliza la regla de nombres de ValidadorAFD para mantener compatibilidad entre fases. Rechaza epsilon en sus distintas representaciones, comprueba símbolos de un carácter y verifica cada origen, símbolo y destino. A diferencia del AFD, no considera error una transición ausente ni un conjunto destino vacío.

10.4 Clase ConvertidorAFND

El convertidor recorre únicamente los subconjuntos alcanzables desde {q0}. Dos recorridos que produzcan el mismo conjunto reutilizan el mismo identificador. El método _crear_identificador extiende la nomenclatura después de Z y mostrar_equivalencias conserva el orden de descubrimiento para facilitar la explicación del proceso.

10.5 Coordinación del menú

preparar_nuevo_automata concentra la validación y conversión automática. obtener_afd_operativo decide qué AFD utilizar. El ciclo principal captura EOF para finalizar sin traceback y KeyboardInterrupt para cancelar una operación sin perder la sesión. Las rutas relativas parten del directorio desde el cual se inicia Python y también se aceptan rutas absolutas entre comillas coincidentes.

XI. COMPLEJIDAD Y DECISIONES DE INGENIERÍA

Tabla 7: Complejidad de las operaciones nuevas

Operación	Complejidad	Fundamento
Validar AFND	$O(\|Q\| + \|\Sigma\| + \|F\| + D)$	D es la suma de destinos almacenados en todas las transiciones.
Calcular transición de macroestado	$O(\|S\| + D_s)$	Se recorren los estados del subconjunto S y sus destinos para un símbolo.

Conversión completa	O(2^|Q| · |Σ| · |Q|)	En el peor caso son alcanzables todos los subconjuntos de Q.
Espacio de conversión	O(2^|Q| · |Q|)	Se almacenan macroestados, equivalencias y transiciones generadas.
Evaluar cadena	O(|w|)	El AFD equivalente realiza una consulta de diccionario por símbolo.

El crecimiento exponencial es una propiedad del método de subconjuntos y no un error de implementación. Para autómatas académicos su costo es manejable; para definiciones grandes conviene considerar límites de entrada o advertencias antes de convertir.

La solución evita duplicar el simulador: el AFND conserva su modelo y el AFD generado concentra la ejecución. También protege la sesión anterior durante cargas fallidas, valida nuevamente antes de simular y conserva el historial cuando una conversión existente todavía es válida.

XII. DEPURACIÓN Y VERIFICACIÓN

La revisión técnica compiló todos los módulos, ejecutó pruebas unitarias y recorrió el menú con entradas controladas. Se comparó la aceptación del AFND mediante una simulación independiente de conjuntos contra el AFD convertido para todas las cadenas binarias de longitud cero a seis y cuatro configuraciones de estados finales. El resultado fue equivalente en 508 casos.

Tabla 8: Casos comprobados durante la depuración

Prueba	Resultado
Sintaxis de todos los módulos	Compilación completada sin errores.
Barra vertical y conjunto vacío	Se aceptan q0|q1, ∅ y {}; se rechazan barras vacías, duplicados y mezclas con ∅.
AFND sin transiciones	Se considera válido y produce un AFD con macroestado vacío alcanzable.
Equivalencia de lenguajes	508 comparaciones entre simulación directa del AFND y el AFD generado.
Símbolo externo	Se reporta el error y no se contamina el historial.
Flujo real del menú	Carga, conversión automática, equivalencias, consulta, evaluación e historial completados.

Durante el recorrido se detectó un cierre inesperado en consolas de Windows cuya codificación no podía representar el símbolo ∅. main configura ahora la salida como UTF-8 con sustitución segura cuando el entorno permite reconfigurarla. Después del ajuste, las cinco pruebas automatizadas y el recorrido completo del menú finalizaron correctamente.

XIII. DIAGRAMA DE FLUJO

El flujo de la Fase 2 prepara la conversión en el mismo momento en que el AFND se incorpora a la sesión. La opción 7 no es un requisito previo para evaluar; funciona como confirmación y mecanismo de reconstrucción.

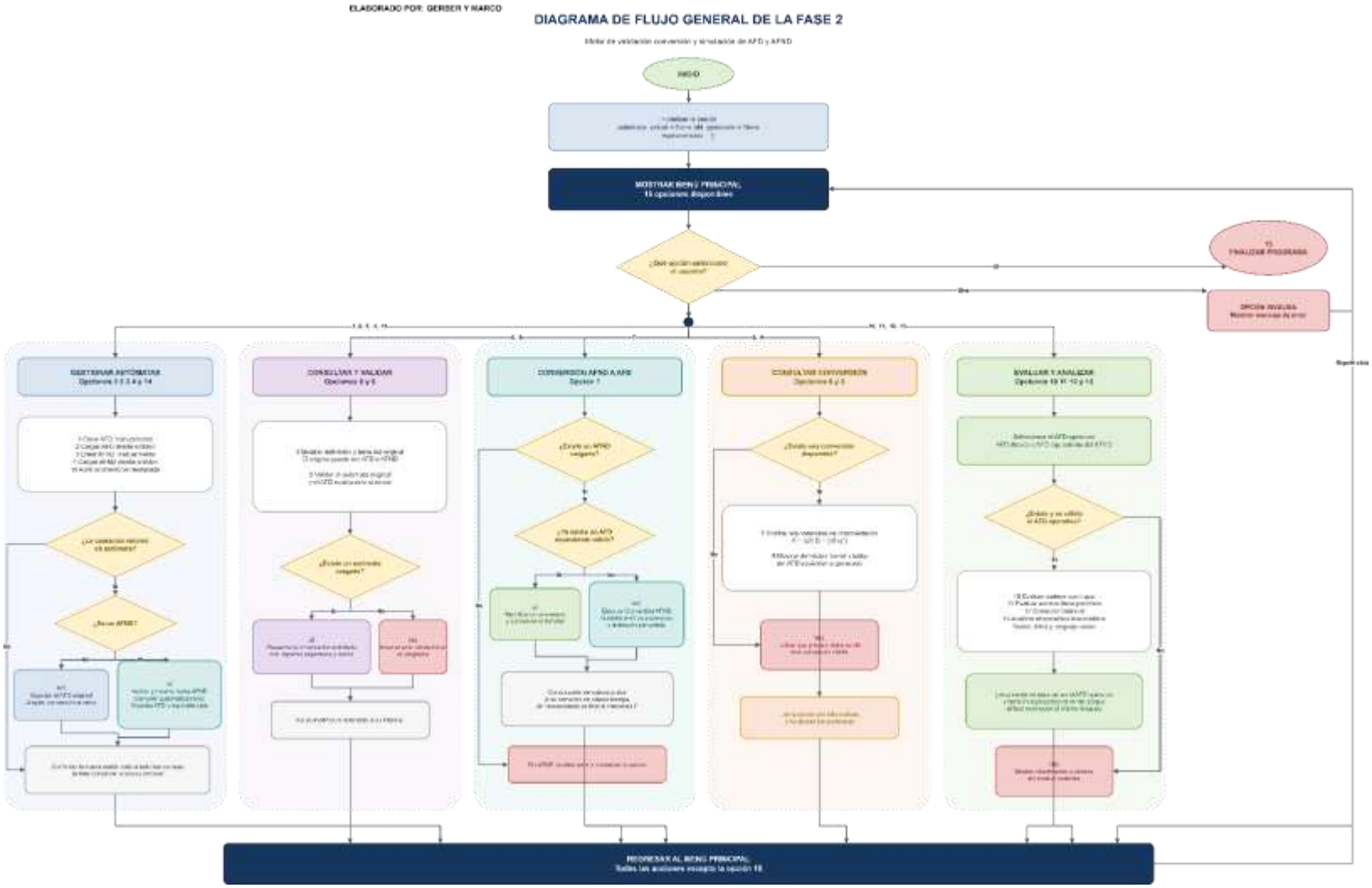


Figura 1: diagrama de flujo completo actualizado con todas las partes de la fase 2. Este es una vista general y representativa de la lógica del programa. Para visualizarlo mejor, presiona la imagen.

XIV. CONCLUSIONES

1. La Fase 2 incorpora el modelo no determinista sin romper la separación de responsabilidades construida para el AFD. Los destinos múltiples se representan mediante conjuntos y se ingresan con una barra vertical; la ausencia de destinos se representa con ∅ o {}. Esta diferencia queda contenida en las clases del AFND y no altera la semántica del simulador determinista.

2. La conversión automática es parte del flujo de creación y carga. El programa valida primero el AFND, construye los macroestados alcanzables, determina los estados finales del AFD y confirma la validez del resultado. La opción 7 sigue disponible para consultar o repetir el proceso, pero no es necesaria para habilitar las evaluaciones cuando la conversión inicial fue correcta.

3. Las pruebas se ejecutan sobre el AFD operativo y sus resultados son válidos para ambos autómatas porque preservan el mismo lenguaje. La tabla de equivalencias y las vistas separadas del original y del generado permiten comprobar el proceso antes de evaluar cadenas.

4. La depuración confirmó el manejo de conjuntos vacíos, destinos múltiples, AFND parciales, símbolos externos y conservación del historial. También se corrigió la salida Unicode en Windows para evitar que la representación de ∅ interrumpa la sesión.

Universidad Rafael Landívar

Facultad de Ingeniería

Licenciatura en Ingeniería en Informática y Sistemas

Lenguajes Formales y Autómatas Sección: 01

Docente: Ing. Cesar Rojas

PROYECTO — PARTE 3

MOTOR DE PARSING, VALIDACIONES ESTRUCTURALES Y SIMULACIÓN DE AUTÓMATAS DE PILA DETERMINISTAS Y NO DETERMINISTAS (APD/APND): ANÁLISIS Y DISEÑO

Estudiantes: Gerber Mariano Péres Gómez,
Marco Fabrizzio Donadio Castellanos

Carné: 1165825, 107692

Guatemala, 21 de octubre de 2026

TABLA DE CONTENIDO — PARTE 3

I. INTRODUCCIÓN 1
II. ALCANCE Y ARQUITECTURA DE LA FASE 3 1
2.1 Separación de responsabilidades 1
III. MODELO FORMAL Y OPERACIONES DE PILA 1
3.1 Configuración instantánea 2
3.2 Efecto de una transición 2
3.3 Criterio de aceptación 2
IV. ACCIONES DEL MENÚ PARA APD Y APND 2
4.1 Tabla de transición 3
4.2 Conservación de la sesión 3
V. DATOS DE ENTRADA Y FORMATO DE ARCHIVOS 3
5.1 Archivo de ejemplo: lenguaje $a^n b^n$, con $n \geq 1$ 4
5.2 Sintaxis y separadores 4
VI. CONDICIONES Y VALIDACIONES 4
6.1 Parsing y validación semántica 5
6.2 Cadena, ciclos y presupuesto 5
VII. VARIABLES Y ESTRUCTURAS DE DATOS 5
7.1 Representación interna de la pila 6
7.2 Memoria de ramas y eventos 6
VIII. ALGORITMO DE SIMULACIÓN 6
8.1 Aceptación y cierre de ramas 7
8.2 Por qué se compara la pila completa 7
IX. TRAZAS Y EXPLICACIÓN DE RESULTADOS 7
9.1 Cadena aceptada: aabb 7
9.2 Cadena rechazada: aab 8
9.3 Entrada inválida y límite de exploración 8
9.4 Presentación de recorridos extensos 8
X. COMPONENTES, COMPLEJIDAD Y DECISIONES DE INGENIERÍA 8
10.1 Implementación de los componentes 8
10.2 Costos de las operaciones 9
10.3 Análisis estructural y alcance de sus conclusiones 9
XI. DEPURACIÓN Y VERIFICACIÓN 9
11.1 Reproducción de la verificación 10
11.2 Límites de la evidencia 10
XII. DIAGRAMAS DE FLUJO 11
12.1 Carga, validación y operaciones de consola 11
12.2 Exploración de configuraciones y veredicto 12
XIII. CONCLUSIONES DE LA FASE 3 13
Fuentes de esta sección 13

I. INTRODUCCIÓN

La Fase 3 incorpora la representación, carga, validación y simulación de Autómatas de Pila Deterministas (APD) y No Deterministas (APND). La pila agrega memoria a la ejecución: una transición depende del estado actual, de un símbolo de entrada o EPS y del símbolo situado en el tope. Su aplicación modifica el estado, puede consumir entrada y sustituye el tope por una secuencia de símbolos.

El programa se ejecuta en consola y permite observar cada configuración, las operaciones de pila y las alternativas exploradas. La aceptación exige consumir toda la cadena y alcanzar un estado final; no exige vaciar la pila. En un APND basta con encontrar una rama que acepte, mientras que el rechazo requiere agotar las posibilidades sin aceptación.

II. ALCANCE Y ARQUITECTURA DE LA FASE 3

El objetivo es procesar cualquier APD o APND válido expresado con el formato del proyecto. Se incluye ingreso manual y por archivo, consulta de la séptupla y tabla, evaluación individual y por lote, historial, análisis del grafo y control del presupuesto de exploración. La implementación utiliza Python y su biblioteca estándar.

Tabla 1. Componentes de la fase 3

Componente	Responsabilidad
AutomataPila	Almacena la séptupla, el tipo, las alternativas de transición, la validación y el historial; presenta la tabla.
CargadorAP	Interpreta el archivo o la entrada manual y entrega un autómata únicamente después de validarlo.
ValidadorAP	Comprueba referencias, alfabetos, reemplazos y las restricciones adicionales de un APD.
SimuladorAP	Explora configuraciones por niveles, controla ciclos y límites, determina el resultado y presenta las trazas.
main	Coordina la sesión AP, el menú, los lotes, el historial y el límite configurable.

2.1 Separación de responsabilidades

El modelo conserva la definición; el cargador interpreta la sintaxis; el validador decide si la estructura es admisible; el simulador ejecuta el autómata. La interfaz coordina estas operaciones. El AP se simula directamente con su pila y no se somete a conversión a un autómata finito.

III. MODELO FORMAL Y OPERACIONES DE PILA

El autómata se representa mediante la séptupla $\mathbf{M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)}$. El campo TIPO indica si se deben exigir las condiciones de APD o permitir las alternativas de APND. TIPO y NOMBRE son metadatos de la implementación, adicionales a los siete componentes formales.

Tabla 2. Componentes de la séptupla

Símbolo	Significado y representación
Q	Conjunto de estados: set[str]. Debe contener al estado inicial.
Σ	Alfabeto de entrada: set[str]. Cada símbolo tiene exactamente un carácter; puede ser vacío para un AP que solo use EPS.
Γ	Alfabeto de pila: set[str]. Permite símbolos de varios caracteres, como Z0; contiene el símbolo inicial.
δ	Diccionario con clave (estado, entrada, tope) y lista ordenada de alternativas (destino, reemplazo).
q_0 y Z_0	Estado inicial y símbolo inicial de pila: str, con pertenencia a Q y Γ respectivamente.
F	Conjunto de estados finales: set[str], subconjunto de Q; se permite que sea vacío.

3.1 Configuración instantánea

Una configuración es **(q, i, P)**, donde q es el estado, i la posición de la siguiente entrada y P el contenido completo de la pila. La raíz es (q_0, 0, [Z_0]). El tope se presenta a la izquierda: en [A,Z0], A es el símbolo que puede consultarse o retirarse.

3.2 Efecto de una transición

Para una pila Xβ, una alternativa (q′, γ) sustituye X por γ y conserva β. Si consume un símbolo, i aumenta en uno; si la entrada es EPS, i permanece igual. En la notación del archivo, A;Z0 representa dos símbolos y deja A como nuevo tope. EPS como reemplazo equivale a la secuencia vacía: retira X y no agrega símbolos.

Ejemplos: reemplazar Z0 por A;Z0 apila A; reemplazar A por EPS desapila A; reemplazar A por A conserva el contenido. Estas operaciones se realizan sobre una pila propia de cada configuración. Las ramas pueden compartir internamente partes inmutables de la pila sin interferirse.

3.3 Criterio de aceptación

Se acepta si i = |w| y q pertenece a F. La comprobación se hace antes de intentar otra transición. Una pila vacía no implica aceptación: si no se cumple el criterio anterior, esa rama termina porque no dispone de un tope al cual aplicar una transición.

IV. ACCIONES DEL MENÚ PARA APD Y APND

La sesión utiliza ap_actual para identificar el autómata de pila activo. Las siguientes opciones corresponden a su operación en la interfaz actual. Los números se conservan tal como aparecen en el programa; las operaciones exclusivas de otros modelos quedan fuera del alcance de este apartado.

Tabla 3. Opciones aplicables a la fase 3

Opción	Acción	Comportamiento
15	Crear AP manualmente	Solicita nombre, tipo, estados, alfabetos, iniciales, finales y transiciones; valida antes de activar.
16	Cargar AP desde .txt	Interpreta y valida el archivo completo; una carga fallida conserva la sesión anterior.
5	Consultar definición y tabla	Presenta la séptupla y organiza δ por pareja (estado, tope) y símbolo de entrada.
6	Validar AP	Informa si la estructura es válida y muestra los errores encontrados.
10	Evaluar una cadena	Enter representa la cadena vacía. Muestra resultado, recorrido, cierres de ramas y contadores.
11	Evaluar un lote	Lee una cadena por línea y reinicia pila y presupuesto para cada evaluación.
12	Consultar historial	Muestra los resúmenes de evaluaciones del AP activo, incluidos los resultados inconclusos.
13	Análisis estructural	Consulta alcanzabilidad en el grafo de estados; informa las limitaciones de este análisis.
17	Ajustar límite de exploración	Recibe un entero positivo; Enter conserva el valor previo. El valor inicial es 10000 configuraciones.
18	Salir	Finaliza el ciclo principal.

4.1 Tabla de transición

Cada fila corresponde a (q, X), con q en Q y X en Γ. Las columnas representan los símbolos de Σ; se añade EPS cuando alguna transición lo usa como entrada. Una celda muestra (destino, reemplazo), varias alternativas en APND o un guion si no existe transición. La ausencia de transición no invalida el autómata: δ puede ser parcial.

4.2 Conservación de la sesión

La activación ocurre después de una carga o creación válida. Una definición con errores no reemplaza el AP anterior. Las evaluaciones se registran en el historial del objeto activo; ENTRADA_INVALIDA se excluye porque no inicia una simulación. Al cambiar de autómata se cambia también el historial consultado.

V. DATOS DE ENTRADA Y FORMATO DE ARCHIVOS

Tabla 4. Información recibida por el sistema

Dato	Ingreso y tipo interno
NOMBRE / TIPO	Texto no vacío para el nombre; APD o APND para el tipo. Ambos se almacenan como str.
ESTADOS / ALFABETO / ALFABETO_PILA / FINALES	Listas separadas por comas que se convierten en conjuntos. ALFABETO y FINALES pueden quedar vacíos; no se aceptan duplicados.
INICIAL / INICIAL_PILA	Un estado y un símbolo de pila, respectivamente; se almacenan como str.
TRANSICIONES	Clave de tres str; alternativas con destino str y reemplazo tuple[str, ...].
Cadena / lote / límite	Cadena str; una cadena por línea en el lote; presupuesto int positivo por evaluación.

5.1 Archivo de ejemplo: lenguaje $a^n b^n$, con $n \geq 1$

```
NOMBRE=APD_ANBN
TIPO=APD
ESTADOS=q0,q1,q2
ALFABETO=a,b
ALFABETO_PILA=Z0,A
INICIAL=q0
INICIAL_PILA=Z0
FINALES=q2
TRANSICIONES:
q0,a,Z0,q0,A;Z0
q0,a,A,q0,A;A
q0,b,A,q1,EPS
q1,b,A,q1,EPS
q1,EPS,Z0,q2,Z0
```

Las primeras a agregan símbolos A. Cada b retira uno; al recuperar Z0 se puede pasar a q2 mediante EPS. En esta definición la cadena vacía no se acepta, porque q0 no es final y no tiene transición EPS inicial.

5.2 Sintaxis y separadores

La transición tiene cinco campos: origen,entrada,tope,destino,reemplazo. Para agregar otra alternativa a la misma clave se utiliza |destino,reemplazo. Por ejemplo, q0,a,Z0,q1,Z0|q2,A;Z0 conserva dos alternativas en un APND. El punto y coma separa símbolos de reemplazo; no se interpreta Z0 como dos caracteres.

Se aceptan archivos .txt con UTF-8 y BOM opcional. Los ocho encabezados deben aparecer una sola vez antes de TRANSICIONES:. El cargador reconoce sus nombres sin exigir un orden fijo, aunque el ejemplo conserva el orden de escritura del proyecto. No se admiten líneas vacías en la definición. En un

archivo de cadenas, en cambio, una línea vacía sí representa ε; solo se eliminan CR y LF, no los espacios de la cadena.

VI. CONDICIONES Y VALIDACIONES

Tabla 5. Reglas para aceptar una definición

Condición	Criterio aplicado
Integridad de estados	q_0 pertenece a Q; F es subconjunto de Q; todo origen y destino declarado pertenece a Q.
Integridad de la pila	Z_0, cada tope y cada símbolo de reemplazo pertenecen a Γ. EPS no se admite como tope.
Entrada válida	Cada símbolo de Σ tiene un carácter. La entrada de una transición pertenece a Σ o es EPS.
Uso de EPS	No pertenece a Σ ni a Γ. Como reemplazo aparece solo; se rechazan A;EPS y EPS;Z0.
APD: alternativa única	Cada clave (estado, entrada, tope) tiene como máximo una alternativa distinta.
APD: conflicto EPS/consumo	Si existe δ(q, EPS, X), no puede existir δ(q, a, X) para ningún a de Σ.
APND: alternativas	Se permiten varios destinos y reemplazos y coexistencia de EPS con consumo; todas las alternativas deben ser estructuralmente válidas.
Duplicados y campos	Se detectan encabezados repetidos, campos vacíos indebidos, separadores incorrectos y alternativas exactamente duplicadas.
Función parcial	Solo se escriben las transiciones existentes. No se exige completar todas las combinaciones.

6.1 Parsing y validación semántica

El cargador utiliza las expresiones **^([A-Z_]+)=(.*)$** para encabezados y **^TRANSICIONES:$** para el marcador. Los tokens se comprueban con **^[^\s,;|]+$**, que excluye espacios y separadores. Luego se descompone cada transición y se verifica la pertenencia de sus referencias. El reconocimiento de la forma de una línea no sustituye su validación semántica.

6.2 Cadena, ciclos y presupuesto

Antes de crear la configuración inicial se comprueba que el AP esté validado, que cada carácter de la cadena pertenezca a Σ y que el límite sea un entero positivo. Una entrada inválida produce un diagnóstico sin recorrido. Durante la simulación solo se cierra por ciclo una alternativa que repite estado, posición y pila completa en su propia rama.

El límite es una protección exigida por el enunciado de Fase 3, en el apartado Restricciones, página 6. El valor de 10000 y su ajuste por menú son decisiones de implementación. Alcanzar el presupuesto no demuestra rechazo: las alternativas omitidas obligan a informar un resultado inconcluso si no se encuentra aceptación.

VII. VARIABLES Y ESTRUCTURAS DE DATOS

Tabla 6. Datos persistentes y de simulación

Variable o estructura	Tipo / función
ap_actual / limite_ap	Referencia al AutomataPila activo y entero con el presupuesto por cadena.
estados / alfabeto / alfabeto_pila / estados_finales	Conjuntos que representan Q, Σ, Γ y F. Facilitan las consultas de pertenencia.
transiciones	dict[(str,str,str), list[(str,tuple)]]. Conserva el orden de las alternativas.
es_valido / historial	bool que refleja la validación y list de resúmenes con cadena, resultado, motivo, tipo y contadores.
_NodoPila / cache_pilas	Nodo con símbolo y resto. El diccionario reutiliza un nodo para cada pareja (símbolo, resto).
cola	collections.deque de configuraciones pendientes. Se retira por la izquierda y se agrega por la derecha.
nodos_por_id	dict que relaciona cada identificador con su nodo de exploración y el enlace a su padre.
ids_por_configuracion	Índice de (estado, posición, pila) a los nodos donde apareció. La coincidencia se confirma contra la ascendencia de la rama.
id / id_padre / id_rama / profundidad	Enteros y padre opcional que identifican cada configuración, su origen, su rama y el nivel de exploración.
admitidas / procesadas / generadas / podadas_limite	Contadores de nodos admitidos, nodos retirados de la cola, alternativas examinadas y alternativas omitidas por presupuesto.
ResultadoSimulacion	Agrupa veredicto, motivo, cadena, límite, contadores, eventos, nodo aceptador y camino testigo.

7.1 Representación interna de la pila

La simulación utiliza nodos enlazados compartidos que no modifica después de crearlos. Un reemplazo construye o reutiliza su prefijo y conserva el resto de la pila. Esta representación reduce las copias entre configuraciones. El camino testigo público convierte las pilas a tuplas; la salida las imprime del tope hacia el fondo.

7.2 Memoria de ramas y eventos

Cada nodo guarda un enlace al padre, en lugar de copiar todos sus ancestros. Los eventos registran expansiones, aceptación, bloqueo, pila vacía, ciclos, pendientes y omisiones por límite. El historial del menú conserva resúmenes; no guarda el objeto completo de exploración de cada evaluación.

VIII. ALGORITMO DE SIMULACIÓN

El mismo motor resuelve APD y APND mediante búsqueda por niveles (BFS). En un APD válido existe como máximo una continuación aplicable; en un APND se generan configuraciones hijas para las distintas alternativas. El recorrido por niveles evita que una rama profunda impida explorar las demás.

1. Validar las condiciones previas. Comprobar el indicador de validez, los símbolos de entrada y el presupuesto. Si fallan, devolver ENTRADA_INVALIDA sin crear la raíz.

2. Inicializar la búsqueda. Crear (q_0, 0, [Z_0]), asignarle identificador 0 e incorporarla a la cola y a los índices. La raíz cuenta como una configuración admitida.

3. Retirar una configuración. Extraer el primer nodo de la cola y aumentar el contador de procesadas. Si consumió toda la entrada y su estado es final, reconstruir el testigo y aceptar.

4. Encontrar alternativas. Si la pila está vacía, cerrar la rama. En otro caso, consultar las transiciones EPS y después las del siguiente símbolo, si queda entrada. Sin alternativas, registrar el bloqueo.

5. Calcular cada hija. Sustituir el tope, actualizar el estado y avanzar la posición únicamente cuando se consume entrada. Cada alternativa conserva la independencia de su pila.

6. Controlar ciclos exactos. Buscar la configuración resultante en el índice y confirmar si apareció entre los ancestros de la rama. Si se repite, registrar el ciclo y omitir únicamente esa continuación.

7. Aplicar el presupuesto. Si admitidas es menor que el límite, crear y encolar la hija. Si no, registrar la alternativa omitida y continuar procesando los nodos que ya estaban admitidos.

8. Resolver el agotamiento. Si la cola termina sin aceptación y hubo omisiones por límite, devolver DETENIDA_POR_LIMITE. Si no hubo omisiones, devolver RECHAZADA.

8.1 Aceptación y cierre de ramas

La existencia de una rama aceptadora basta para terminar con ACEPTADA, incluso si antes se omitió alguna alternativa por límite. Las configuraciones que quedan en la cola se identifican como pendientes por aceptación de otra rama; no se presentan como rechazadas. La pila, el presupuesto y los índices se reinician para cada cadena del lote.

8.2 Por qué se compara la pila completa

Dos configuraciones con igual estado y posición pueden tener futuros distintos si cambia la pila. Tampoco basta con comparar el tope. La detección de ciclo exige la coincidencia completa dentro del mismo camino. Un ciclo EPS que aumenta continuamente la pila no repite configuraciones exactas y se contiene mediante el presupuesto.

IX. TRAZAS Y EXPLICACIÓN DE RESULTADOS

La traza registra paso, nodo, padre y rama; estado y entrada restante antes de la transición; pila anterior, entrada consumida o EPS, tope leído, destino, reemplazo y pila posterior. Una marca TESTIGO

permite distinguir el camino aceptador. Los cierres de ramas y las alternativas omitidas se muestran en tablas separadas.

9.1 Cadena aceptada: aabb

La siguiente tabla resume el recorrido del APD_ANBN de la sección 5.1. La pila anterior pertenece a la configuración de origen, no a la de destino. Todas las transiciones de este ejemplo siguen una única rama.

Tabla 7. Evolución de la pila para aabb

Paso	Estado	Pendiente	Pila antes	Lee	Destino	Pila después
0	q0	aabb	[Z0]	—	—	—
1	q0	aabb	[Z0]	a	q0	[A,Z0]
2	q0	abb	[A,Z0]	a	q0	[A,A,Z0]
3	q0	bb	[A,A,Z0]	b	q1	[A,Z0]
4	q1	b	[A,Z0]	b	q1	[Z0]
5	q1	ε	[Z0]	EPS	q2	[Z0]

Después del paso 5 se consumió aabb y q2 pertenece a F; por ello el resultado es ACEPTADA, aunque Z0 permanezca en la pila. El recorrido admite y procesa seis configuraciones contando la inicial.

9.2 Cadena rechazada: aab

La cadena aab realiza tres transiciones y llega a (q1, 3, [A,Z0]). Ya no queda entrada, q1 no es final y no existe transición EPS con tope A. La rama cierra sin aceptar y el resultado es RECHAZADA. Sí existe traza: muestra los pasos efectivamente realizados y el motivo del cierre. En APND se deben considerar también las demás ramas antes de concluir rechazo.

9.3 Entrada inválida y límite de exploración

Para x, que no pertenece a Σ = {a,b}, el resultado es ENTRADA_INVALIDA: no se inicia recorrido. Para aabb con límite 3, se admiten la raíz y las dos primeras hijas; la siguiente alternativa queda omitida. El resultado es DETENIDA_POR_LIMITE, no rechazo. La traza diferencia una transición ejecutada de una alternativa potencial no admitida.

9.4 Presentación de recorridos extensos

Cuando un grupo supera 50 eventos, la consola muestra los primeros 25 y los últimos 10 con un aviso explícito. El detalle estructurado se conserva en ResultadoSimulacion.pasos durante esa evaluación. Este resumen de pantalla no elimina eventos del resultado, pero no equivale a una exportación permanente del recorrido.

X. COMPONENTES, COMPLEJIDAD Y DECISIONES DE INGENIERÍA

10.1 Implementación de los componentes

AutomataPila reúne los datos del AP y ofrece agregar_alternativa, obtener_alternativas, consulta formal, tabla e historial. La incorporación de una alternativa invalida la marca previa de validación. **CargadorAP** separa encabezados, listas, reemplazos y alternativas; acumula errores y evita activar una definición parcialmente válida.

ValidadorAP.validar devuelve la validez y una lista de errores; aplica las reglas de determinismo únicamente cuando TIPO es APD. **SimuladorAP.evaluar** constituye el núcleo silencioso y devuelve ResultadoSimulacion. mostrar_traza presenta el resultado, mientras que evaluar_y_mostrar combina ambas operaciones para el menú.

10.2 Costos de las operaciones

Sean K las claves de transición, A las alternativas almacenadas, R el total de símbolos de reemplazo, N las configuraciones admitidas, G las alternativas examinadas y H la altura máxima de pila. El presupuesto acota N, pero no fija por sí solo la longitud de las pilas ni el tamaño total del texto mostrado.

Tabla 8. Costos y factores que los determinan

Operación	Análisis
Validación semántica	Recorre los alfabetos, finales, K claves, A alternativas y R símbolos. Con pertenencia en conjuntos, el costo esperado es proporcional a esas cantidades.
Tabla de transición	Visita $\|Q\| \cdot \|\Gamma\|$ filas y $\|\Sigma\|$ columnas, más EPS si corresponde. El formato también depende del texto de las alternativas.
Reemplazo de pila	Para r símbolos insertados, realiza O(r) operaciones esperadas de caché. Compartir el resto evita copiar toda la pila.
Control de ciclos	El índice localiza coincidencias; comprobar ascendencia recorre enlaces al padre. Su costo depende de la cantidad de coincidencias y de la profundidad; no es siempre constante.
Exploración y memoria	Guarda N nodos, índices, pilas compartidas y eventos asociados a G alternativas. En APND el número de ramas puede crecer exponencialmente con la profundidad.
Testigo y presentación	Reconstruir d pasos requiere seguir d padres; materializar sus pilas puede costar $O(d \cdot H)$. El resumen limita la salida visible, no todos los datos internos.

10.3 Análisis estructural y alcance de sus conclusiones

La opción 13 recorre el grafo de estados ignorando condiciones de entrada y pila. Si ningún estado final es alcanzable ni siquiera en ese grafo, el lenguaje es vacío. Si existe un final conectado, eso no basta para afirmar que el lenguaje sea no vacío: el camino puede ser imposible de ejecutar. Este análisis no es un procedimiento completo de decisión de vaciedad para AP.

XI. DEPURACIÓN Y VERIFICACIÓN

La revisión del 21 de octubre de 2026 utilizó una copia del código actual y de sus archivos de prueba, sin modificar el proyecto original. Se comprobó la sintaxis de los 13 módulos principales. La suite completa ejecutó 279 pruebas y finalizó correctamente. La tabla siguiente se concentra exclusivamente en los comportamientos de la Fase 3.

Tabla 9. Casos de verificación de la fase 3

Caso	Resultado comprobado
APD_ANBN con aabb	ACEPTADA; seis configuraciones, incluida la raíz; pila final [Z0].
APD_ANBN con aab	RECHAZADA; cuatro configuraciones; cierre en q1 sin entrada pendiente.
APD_ANBN con x	ENTRADA_INVALIDA; no se crean configuraciones ni eventos.
APD_ANBN con aabb y límite 3	DETENIDA_POR_LIMITE; tres configuraciones y una alternativa omitida.
Carga y validación	Formatos, referencias, duplicados, EPS y determinismo.
Simulación y ramas	Ciclos, pila vacía, aceptación, límite e independencia de ramas.
Trazas	Aceptación, rechazo, cierres, omisiones y ramas pendientes.
Consola	Ingreso manual, archivo, lote, historial, sesión y límite.

11.1 Reproducción de la verificación

Desde la raíz del proyecto, los siguientes comandos permiten verificar la sintaxis y ejecutar las pruebas. La aplicación se inicia con el mismo intérprete de Python utilizado para comprobarla. Los comandos no requieren instalar bibliotecas de autómatas.

```
python -X utf8 -m compileall -q .
python -X utf8 -m unittest discover -s tests -q
python -X utf8 main.py
```

11.2 Límites de la evidencia

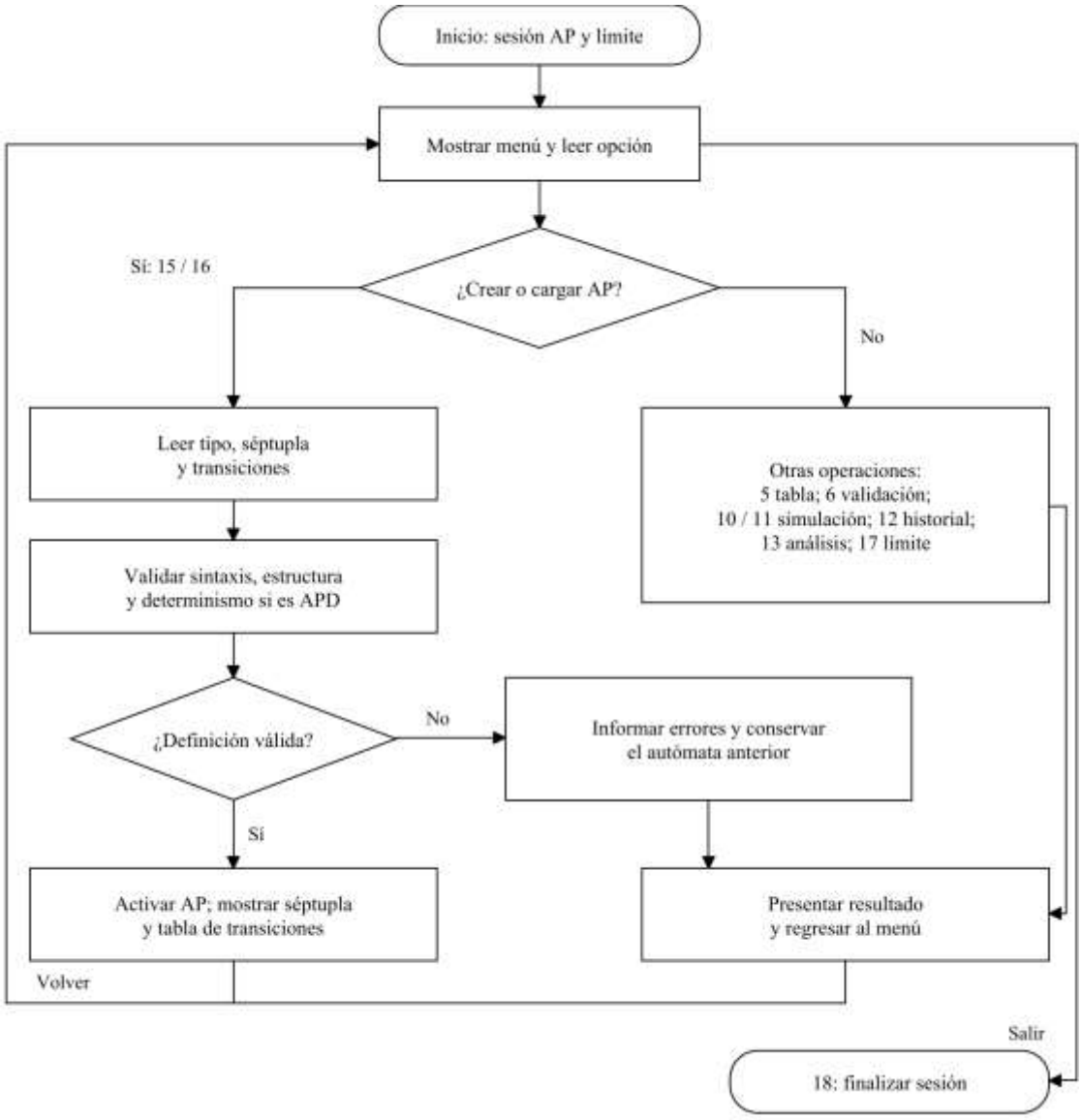
Las pruebas respaldan los casos comprobados; no demuestran corrección para toda definición posible. Un resultado inconcluso puede repetirse con un presupuesto mayor, sin garantizar un veredicto concluyente cuando existen infinitas configuraciones distintas. El resumen de trazas y el análisis de grafo mantienen las limitaciones descritas en las secciones anteriores.

XII. DIAGRAMAS DE FLUJO

12.1 Carga, validación y operaciones de consola

El flujo muestra la incorporación del AP y las operaciones que actúan sobre la sesión de pila. El archivo editable Diagrama_Fase3.drawio contiene esta vista y la vista de simulación de la página siguiente. Las operaciones concluidas regresan al menú, conservando el autómata activo.

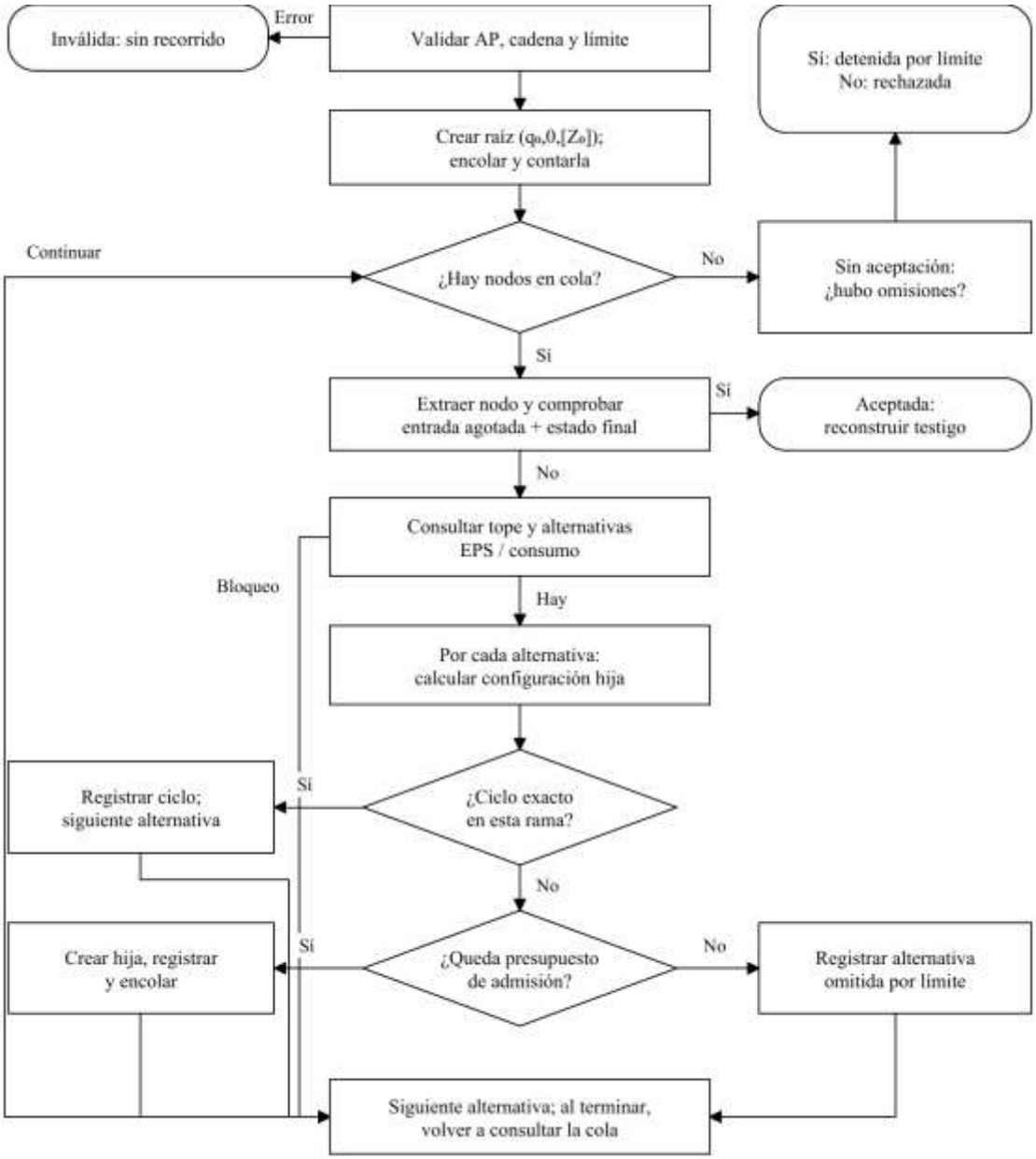
Figura 1. Flujo de carga, validación y operaciones sobre el AP activo.



12.2 Exploración de configuraciones y veredicto

La simulación conserva las alternativas omitidas por presupuesto como pendientes de exploración. Solo informa rechazo al terminar sin aceptación y sin omisiones por límite. Los ciclos exactos se cierran por rama, sin descartar por ello otras alternativas.

Figura 2. Motor común APD/APND; la pila vacía sin aceptación cierra la rama.



XIII. CONCLUSIONES DE LA FASE 3

1. Representación formal. La séptupla permite expresar de manera explícita estados, entrada, pila y transiciones. El modelo distingue los símbolos de entrada de un carácter de los tokens de pila, que pueden contener varios caracteres. La sintaxis con comas, punto y coma y barra vertical evita confundir campos, reemplazos y alternativas.

2. Validación del comportamiento declarado. Un APD se admite únicamente cuando no hay varias alternativas para una clave ni conflicto entre EPS y consumo para el mismo estado y tope. Un APND conserva sus alternativas válidas. La ausencia de una transición no es un error de integridad, sino una posible causa de bloqueo durante la ejecución.

3. Gestión de pila y ramas. El motor utiliza configuraciones con estado, posición y pila completa. Las pilas compartidas sin modificación y los enlaces al padre permiten mantener separadas las ramas y reconstruir el camino aceptador sin copiar todo el historial de ancestros en cada paso.

4. Resultados interpretables. Consumir toda la entrada y alcanzar un estado final constituye aceptación, incluso con pila no vacía. Las trazas de rechazo explican el recorrido y el cierre de las ramas. Las entradas inválidas se detectan antes de simular y, por ello, no generan una traza de ejecución.

5. Terminación controlada. La detección de configuraciones repetidas contiene ciclos exactos dentro de una rama. El presupuesto protege frente a exploraciones con configuraciones siempre distintas. Si deja alternativas sin explorar y no aparece un testigo aceptador, se informa un resultado inconcluso, preservando la diferencia entre límite de recursos y rechazo del lenguaje.

6. Uso y verificación. La interfaz reúne creación, carga, consulta, simulación, lotes e historial del AP activo. La verificación ejecutada respalda los casos comprobados, mientras que las limitaciones del resumen de trazas y del análisis de grafo se declaran expresamente. La implementación conserva módulos pequeños y responsabilidades reconocibles para facilitar su explicación técnica.